SE3020 – Distributed Systems

Assignment 1 – Project Plan

AI-Enabled Smart HealthcareAppointment & Telemedicine Platform

Microservices Architecture Plan

Team Size: 3 Members
Duration: 5 Weeks
Tech Stack: Next.js · NestJS · MongoDB · Docker · Kubernetes

1. Recommended Tech Stack

Below is the recommended technology stack, chosen to match your team's existing experience while meeting all assignment requirements.

Layer	Technology	Justification
Frontend	Next.js 15	Your team already knows it. SSR/CSR hybrid, great DX, supports async patterns required by the assignment.
Backend (APIs)	NestJS (Node.js)	Familiar to your team. Built-in module system maps naturally to microservices. TypeScript end-to-end.
Database	MongoDB Atlas (Free Tier)	Flexible schema for healthcare data, free cloud hosting, easy setup for a university project.
Authentication	JWT + Passport.js	Industry-standard, role-based access (Patient/Doctor/Admin). Built-in NestJS support via @nestjs/passport.
API Gateway	Nginx (Reverse Proxy)	Lightweight, routes requests to correct microservice. Simple config inside Docker/K8s.
Video/Telemedicine	Jitsi Meet (Self-hosted or API)	Free and open-source. No API key costs. Easy iframe embed. Your team has experience.
Payments	Stripe (Sandbox)	Sri Lankan payment gateway, sandbox mode for testing. Meets the assignment's local integration requirement.
Notifications	Nodemailer (Email) + Twilio (SMS)	Nodemailer is free with Gmail SMTP. Twilio free trial provides SMS credits.
AI Symptom Checker	OpenAI API (GPT-3.5)	Cheap, fast, good enough for preliminary symptom suggestions. Easy REST integration.
File Storage	Cloudinary or AWS S3 (Free Tier)	Medical report/document uploads. Cloudinary has generous free tier.
Containerization	Docker + Kubernetes (Minikube)	Required by assignment. Minikube runs locally for development and demo.
Inter-service Comms	REST (sync) + RabbitMQ (async)	REST for real-time queries; RabbitMQ for notifications and event-driven flows.

2. Microservices Architecture Design

The system is decomposed into 7 independently deployable microservices, each with its own database (Database-per-Service pattern) and communicating via REST and asynchronous messaging.

2.1 Service Breakdown

Service 1: Auth & User Service

Port: 3001

Database: MongoDB – auth_db

Responsibilities: User registration (Patient, Doctor, Admin), login/logout, JWT token issuance & refresh, role-based access control, profile management, doctor verification (admin-approved).

Key Endpoints:

Method	Endpoint	Description
POST	/api/auth/register	Register new user (patient/doctor)

POST	/api/auth/login	Login and receive JWT tokens
GET	/api/auth/profile	Get current user profile
PUT	/api/auth/profile	Update profile details
GET	/api/auth/doctors	List all verified doctors
PATCH	/api/auth/doctors/:id/verify	Admin verifies a doctor registration

Service 2: Appointment Service

Port: 3002

Database: MongoDB – appointment_db

Responsibilities: Doctor availability/schedule management, appointment booking/modification/cancellation, real-time status tracking, search doctors by specialty.

Key Endpoints:

Method	Endpoint	Description
GET	/api/appointments/doctors/search	Search doctors by specialty/name
GET	/api/appointments/doctors/:id/slots	Get available time slots
POST	/api/appointments	Book a new appointment
PATCH	/api/appointments/:id	Modify/cancel an appointment
PATCH	/api/appointments/:id/accept	Doctor accepts/rejects appointment
GET	/api/appointments/my	Get user's appointments (patient/doctor)

Service 3: Telemedicine Service

Port: 3003

Database: MongoDB – telemedicine_db

Responsibilities: Video session creation and management, Jitsi Meet room provisioning, session recording metadata, prescription issuance during/after consultations.

Key Endpoints:

Method	Endpoint	Description
POST	/api/sessions	Create a video session for an appointment
GET	/api/sessions/:appointmentId	Get session details & join link
PATCH	/api/sessions/:id/end	End a video session
POST	/api/prescriptions	Doctor issues a digital prescription
GET	/api/prescriptions/patient/:id	Get patient's prescriptions

Service 4: Payment Service

Port: 3004

Database: MongoDB – payment_db

Responsibilities: Payment processing via PayHere sandbox, payment status tracking, refund handling, transaction history for admin oversight.

Key Endpoints:

Method	Endpoint	Description
POST	/api/payments/initiate	Initiate payment for an appointment
POST	/api/payments/callback	PayHere webhook callback
GET	/api/payments/:id	Get payment status
GET	/api/payments/history	Transaction history (admin)

Service 5: Notification Service

Port: 3005

Database: MongoDB – notification_db

Responsibilities: Email notifications via Nodemailer, SMS notifications via Twilio, event-driven – listens to RabbitMQ events (appointment booked, consultation complete, payment confirmed).

Key Endpoints:

Method	Endpoint	Description
POST	/api/notifications/send	Send notification (internal use)
GET	/api/notifications/user/:id	Get user's notification history
–	RabbitMQ Consumer	Listens to appointment.booked, session.completed, payment.confirmed events

Service 6: Medical Records Service

Port: 3006

Database: MongoDB – records_db

Responsibilities: Medical report/document upload (to Cloudinary/S3), medical history management, file access control (only patient + assigned doctor can view).

Key Endpoints:

Method	Endpoint	Description
POST	/api/records/upload	Upload medical report/document
GET	/api/records/patient/:id	Get patient's medical records
DELETE	/api/records/:id	Delete a medical record

Service 7: AI Symptom Checker Service (Optional)

Port: 3007

Database: MongoDB – ai_db (stores query history)

Responsibilities: Accept patient symptoms, call OpenAI API for preliminary analysis, suggest doctor specialties, store interaction history.

Key Endpoints:

Method	Endpoint	Description
POST	/api/ai/check-symptoms	Submit symptoms, get AI suggestions
GET	/api/ai/history	Get past symptom check results

2.2 Inter-Service Communication

The services communicate through two patterns:

Synchronous (REST): Used when a service needs an immediate response. For example, the Appointment Service calls the Auth Service to validate a doctor's existence before booking.

Asynchronous (RabbitMQ): Used for event-driven flows where the caller doesn't need to wait. For example, when an appointment is booked, a message is published to RabbitMQ, and the Notification Service consumes it to send email/SMS confirmations.

Key message queue events:

Event	Publisher	Consumers
appointment.booked	Appointment Service	Notification Service, Payment Service
appointment.cancelled	Appointment Service	Notification Service, Payment Service (refund)
payment.confirmed	Payment Service	Appointment Service (confirm booking), Notification Service
session.started	Telemedicine Service	Notification Service
session.completed	Telemedicine Service	Notification Service
prescription.issued	Telemedicine Service	Notification Service, Medical Records Service

2.3 Docker & Kubernetes Setup

Each microservice has its own Dockerfile. Kubernetes deployment uses Minikube locally.

Docker: Each service has a Dockerfile + a root docker-compose.yml for local development (runs all services + MongoDB + RabbitMQ).

Kubernetes: Each service has a Deployment + Service YAML. Use Minikube with an Ingress controller (Nginx) for routing. ConfigMaps for environment variables, Secrets for API keys and DB credentials.

Suggested K8s resource structure:

```
k8s/
  ├── auth-service-deployment.yaml
  ├── appointment-service-deployment.yaml
  ├── telemedicine-service-deployment.yaml
  ├── payment-service-deployment.yaml
```

```
├── notification-service-deployment.yaml
├── records-service-deployment.yaml
├── ai-service-deployment.yaml
├── mongodb-deployment.yaml
├── rabbitmq-deployment.yaml
├── ingress.yaml
└── configmap-secrets.yaml
```

3. Work Distribution (3 Members)

Each member owns specific microservices plus a portion of the frontend. This ensures clear accountability and balanced workload.

	Member A	Member B	Member C
Backend Services	**1.** Auth & User Service **2.** Appointment Service	**1.** Telemedicine Service **2.** Payment Service	**1.** Notification Service **2.** Medical Records Service **3.** AI Symptom Checker
Frontend Pages	Login/Register pages Doctor search & listing Appointment booking flow Admin dashboard	Video consultation room Payment checkout flow Doctor dashboard Prescription view/issue	Patient dashboard Medical records upload/view AI symptom checker UI Notification center
Infrastructure	RabbitMQ setup, database configs, environment management	Kubernetes deployments, CI/CD basics	Docker setup, API Gateway (Nginx), K8s Ingress
Deliverables	Architecture diagram, service interface docs	Workflow diagrams, security section	Individual contributions, README, submission files

4. Five-Week Timeline

This timeline ensures incremental progress with integration checkpoints every week.

Week 1: Foundation & Setup

Day	Tasks
Day 1–2	Set up GitHub repo with monorepo structure. Initialize all NestJS services with boilerplate. Set up Next.js frontend. Create shared types/interfaces package.
Day 3–4	Set up MongoDB Atlas databases (one per service). Configure environment variables. Create docker-compose.yml to run all services locally. Set up a RabbitMQ container.
Day 5–7	Member A: Build Auth Service (register, login, JWT, roles). Member B: Set up Jitsi Meet integration (test iframe embed). Member C: Set up Nodemailer + Twilio test scripts. ALL: Create basic frontend layout (navbar, routing, auth pages).

Week 1 Milestone: All services running in Docker. Auth working end-to-end (register → login → protected route). Basic frontend shell.

Week 2: Core Services Development

Day	Tasks

Day 1–3	Member A: Build Appointment Service (doctor availability, booking, search). Member B: Build Payment Service (PayHere sandbox integration). Member C: Build Medical Records Service (file upload to Cloudinary, CRUD).
Day 4–5	Member A: Frontend – Doctor search page, appointment booking UI. Member B: Frontend – Payment checkout flow. Member C: Frontend – Medical records upload/view pages.
Day 6–7	Integration checkpoint: Test appointment booking → payment flow end-to-end. Set up RabbitMQ event publishing in the Appointment Service. Notification Service starts consuming events.

Week 2 Milestone: Patients can search doctors, book appointments, make payment. Notification sent on booking. Medical records upload working.

Week 3: Telemedicine & Advanced Features

Day	Tasks
Day 1–3	Member B: Build full Telemedicine Service (session creation, Jitsi room provisioning, prescription issuance). Member A: Build Doctor dashboard (view appointments, accept/reject, manage availability). Member C: Build AI Symptom Checker Service (OpenAI integration).
Day 4–5	Member B: Frontend – Video consultation room page. Member A: Frontend – Doctor dashboard, Admin dashboard. Member C: Frontend – AI symptom checker UI, patient dashboard.
Day 6–7	Integration: Full flow – Book → Pay → Join video → Get prescription. All RabbitMQ events flowing correctly. Bug fixes and UI polish.

Week 3 Milestone: Complete patient journey working: register → search doctor → book → pay → video call → receive prescription. AI symptom checker functional.

Week 4: Kubernetes, Integration & Testing

Day	Tasks
Day 1–2	Member A: Write Dockerfiles for all services. Set up Minikube. Create K8s deployment + service YAMLs for each microservice. Member B: Create K8s Ingress (Nginx), ConfigMaps, Secrets. Member C: Deploy MongoDB and RabbitMQ in K8s.
Day 3–4	Deploy all services to Minikube. Test entire application running on Kubernetes. Fix any container networking/configuration issues.
Day 5–7	End-to-end testing of all workflows. Fix bugs. UI responsiveness testing. Security review (JWT validation, role guards, input sanitization).

Week 4 Milestone: Everything running on Kubernetes (Minikube). All features working. No critical bugs.

Week 5: Documentation, Demo & Submission

Day	Tasks

Day 1–2	Write report.pdf: Architecture diagram, service interfaces list, workflow explanations (use sequence diagrams), auth/security section, individual contributions.
Day 3	Record YouTube demo video (3 min per member = 9 min total). Each member demos their services and explains their contribution.
Day 4	Prepare submission files: submission.txt (GitHub + YouTube links), readme.txt (deployment steps), members.txt, report.pdf. Add code appendix to report (copy as text, NOT screenshots).
Day 5	Final review. Check Turnitin similarity < 20%. Zip everything as GroupID_DS-Assignment.zip. Submit.

Week 5 Milestone: All deliverables ready. ZIP submitted.

5. Recommended Repository Structure

Use a monorepo structure for easier development and shared configuration.

healthcare-platform/

```
├── client/                    # Next.js frontend
│   ├── src/app/               # App router pages
│   ├── src/components/        # Reusable UI components
│   ├── src/services/          # API call functions
│   └── Dockerfile
├── services/
│   ├── auth-service/          # NestJS
│   ├── appointment-service/   # NestJS
│   ├── telemedicine-service/  # NestJS
│   ├── payment-service/       # NestJS
│   ├── notification-service/  # NestJS
│   ├── records-service/       # NestJS
│   └── ai-service/            # NestJS
├── k8s/                       # Kubernetes manifests
├── docker-compose.yml         # Local development
├── submission.txt
├── readme.txt
├── members.txt
└── report.pdf
```

6. Key Tips for Success

Start with Docker Compose, not Kubernetes. Get everything working locally with docker-compose first. Only move to K8s in Week 4. This saves massive debugging time.

Use a shared Postman/Insomnia collection. Document all API endpoints in a shared collection so all members can test each other's services easily.

Keep services simple. Each NestJS service should have 3–5 endpoints max. Don't over-engineer. The lecturers want to see proper microservices decomposition, not a monolith split into folders.

Git branching strategy: Use feature branches (feature/auth-service, feature/appointment-ui). Merge to main via pull requests. This keeps the repo clean and shows collaboration.

Record the demo early. Don't leave the YouTube video to the last day. Record it as soon as Week 4 is done, so you have buffer time.

Report code appendix: Copy code as TEXT, not screenshots (the assignment specifically penalizes screenshots). Use a monospace font in the appendix.

Turnitin < 20%: Write the report in your own words. Don't copy from the assignment brief. Paraphrase requirements when referencing them.

The AI Symptom Checker is "optional" but recommended. It's in the title of the assignment ("AI-Enabled"), so implementing it will likely earn extra marks. It's also the easiest service to build – just a wrapper around the OpenAI API.